CSI 4106 Introduction to Artificial Intelligence

Assignment 3: Designing and Diagnosing Neural Networks

Marcel Turcotte

Version: Aug 24, 2026 16:21

🎯 Learning Outcomes

- **Relate** a single sigmoid unit to logistic regression and explain what changes when hidden layers are added.
- **Compare** network depth and width using parameter counts and controlled experiments.
- **Diagnose** underfitting, overfitting, and optimization behaviour from learning curves and validation metrics.
- **Evaluate** the effects of early stopping, activation functions, batch size, L2 regularization, and dropout.
- **Apply** a rigorous training, validation, model-selection, and final-test protocol.

In Assignment 2, you used classical classifiers to predict whether an autonomous underwater vehicle returns before completing its planned survey. You will now study fully connected neural networks on the same data and the same final test partition.

Neural networks are not automatically the best choice for tabular data. Their value here is pedagogical: they make it possible to observe how model capacity, optimization, and regularization interact. The objective is not to search blindly for the largest F1 score. It is to conduct controlled experiments, recognize diagnostic evidence, and justify a final model without allowing the test set to influence its development.

📤 Submission

- **Deadline:** November 16, 2026, at 11 PM.
- **Individual assignment:** This assignment must be completed individually.
- **Submission platform:** Upload both files to Brightspace under Assignment 3.
- **Submission format:**

  1. one Jupyter Notebook (`.ipynb`) containing concise explanations, code, results, tables, and plots;
  2. `asrc_missions_hidden_predictions.csv`, produced in Task 9.

> **!** Important
>
> Your notebook must execute from beginning to end without errors in Google Colab, our reference environment. It may reuse valid local copies of the supplied data, but it must obtain any unavailable source file from the supplied URL without a manual file transfer. It must not require the installation of additional libraries. Retain all cell outputs and plots so that the notebook can be graded without being re-executed.

Requirements

1. Data and common test partition

The response, `mission_interrupted`, is `1` when a vehicle returned before completing its planned survey and `0` when the survey was completed. An interruption may be a precautionary return or an incomplete survey; it does not necessarily represent a catastrophic failure.

The template loads the complete labelled data from Assignment 2:

- asrc_missions_complete.csv

The file contains 6,371 missions, 12 predictors, and the response. The supplied code may use a valid local copy when one is already available and otherwise obtains the file from the URL. Another person must be able to execute the complete notebook in Colab without preparing the data manually.

The template also creates the stratified 80/20 development-test split from Assignment 2 using `random_state=4106`. This produces the common test partition used to compare your classical and neural-network results. The test partition must remain untouched until one final evaluation in Task 8.

The development partition is split again into stratified neural-network training and validation partitions, using 80% for training, 20% for validation, and `random_state=4106`. All architecture, optimization, regularization, stopping, and model-selection decisions must use only these two partitions.

For Task 9, use the independent hidden data:

- asrc_missions_hidden.csv

This file contains 2,124 rows, `mission_id`, and the same 12 predictors, but not the response. Do not inspect or use the hidden data during model development. It is used only after the final internal test evaluation and after the complete model configuration has been locked.

The template supplies the feature groups, fixed data partitions, an unfitted preprocessing factory, and basic checks of the source data. You are responsible for fitting and applying the preprocessor at the correct stages, preparing and validating the NumPy arrays used by Keras, constructing and evaluating the models, and interpreting the results.

2. Common experimental settings

Unless a task explicitly changes a setting, use the following protocol:

- fit the preprocessing operations only on the neural-network training partition, then use the fitted transformer to transform validation data;
- standardize numerical predictors, one-hot encode nominal predictors, ordinally encode `maintenance_grade` in the order `poor`, `fair`, `good`, `excellent`, and pass the binary predictor through unchanged;
- use a one-unit sigmoid output layer, binary cross-entropy, the Adam optimizer with learning rate `0.001`, and a classification threshold of `0.5`;
- use batch size `64`;
- call `keras.utils.set_random_seed(4106)` immediately before constructing each model;
- save the weights that obtain the lowest validation loss during each run;
- report the epoch at which the lowest validation loss occurred and evaluate validation performance using the corresponding saved weights;
- for the positive class `mission_interrupted=1`, report precision, recall, F1, and ROC AUC;
- compute the epoch-level training and validation F1 curves at threshold `0.5`. One correct approach is to record epoch-level Keras precision and recall and calculate their harmonic mean;
- after restoring the lowest-validation-loss weights, recompute the checkpoint metrics from the model's predictions. The checkpoint F1 is not necessarily the largest F1 observed in the epoch history;
- retain the training history needed to plot training and validation loss and positive-class F1.

Use the same training and validation rows for every experiment. Do not resample these partitions between runs. A random seed improves reproducibility, but small numerical differences across hardware and software versions are normal.

The experiments deliberately reuse configurations. Train each distinct configuration once, retain its history and results, and reuse them in later comparisons. A helper function may construct, compile, train, and summarize models, but the experimental choices and interpretation must remain easy to locate.

3. Tasks

Perform the following tasks in order.

1. Prepare and validate the neural-network inputs

The template loads the data, defines the predictor groups, creates both required stratified splits, and supplies an unfitted `make_preprocessor()` factory. Using these objects:

- report the data shape, first five rows, and response counts and proportions;
- report the number of observations and response proportions in the complete data, development partition, neural-network training partition, and validation partition;
- report the number of observations reserved for the test partition without inspecting its predictors or response distribution;
- instantiate a preprocessor, fit it only on the neural-network training predictors, and transform the validation predictors with that fitted preprocessor;
- convert the transformed predictors and the corresponding targets to NumPy arrays suitable for Keras;

- validate and report their shapes and data types, confirm that the values are finite, confirm that the targets contain only `0` and `1`, and confirm that each predictor array remains aligned with its target array;
- report the transformed input width and identify the preprocessing applied to each predictor group;
- explain why the preprocessor must be fitted on neural-network training rows rather than on the complete development partition.

With the required preprocessing, the neural-network training and validation input shapes should be `(4076, 17)` and `(1020, 17)`. A target shape of `(n,)` or `(n, 1)` is acceptable when it is used consistently.

2. Connect logistic regression to a small neural network

Train each of the following for 100 epochs without early stopping:

1. a network with no hidden layer and one sigmoid output unit;
2. a network with one hidden layer containing 8 ReLU units, followed by one sigmoid output unit.

For both models:

- calculate the number of trainable parameters by hand and verify it using Keras;
- plot training and validation loss and F1 against epoch;
- report validation precision, recall, F1, and ROC AUC at the saved checkpoint;
- diagnose underfitting or overfitting using evidence from both the curves and the metrics.

Explain in what sense the first network is equivalent to logistic regression. Also identify at least one practical reason why its result need not exactly match your Assignment 2 logistic-regression result.

The remaining experiments follow a widely used development strategy, often called **Overfit, Then Regularize** [1], [2]. First make overfitting visible, then apply early stopping or explicit regularization and use validation evidence to improve generalization. Deliberate overfitting is a diagnostic stage, not the desired final result, and never justifies using the test partition during development.

3. Study depth and capacity

Train the following ReLU architectures for 150 epochs without early stopping:

Architecture	Hidden layers
Wide	`512`
Intermediate	`90, 90`
Deep	`64, 64, 64`

The supplied preprocessing represents each mission using 17 neural-network inputs. The three architectures were chosen so that, with this input width, their total numbers of trainable parameters are approximately equal. You will calculate and verify these totals. For each architecture:

- calculate and verify the trainable parameter count;
- report the best epoch and validation precision, recall, F1, and ROC AUC at the saved checkpoint;
- plot comparable training and validation curves;
- discuss convergence, generalization, and the onset and severity of overfitting.

Explain what this comparison suggests about depth, while recognizing that a comparison of three architectures cannot establish a universal rule.

Now consider three architectures that each contain 192 hidden units in total: `192`, `96, 96`, and `64, 64, 64`. Calculate their trainable parameter counts, but do not train the first two. Explain why distributing the same total number of hidden units does not provide a controlled comparison of network depth.

4. Evaluate early stopping

Use the unregularized `64, 64, 64` ReLU model from Task 3 as the reference. Train a fresh copy with a maximum of 150 epochs and early stopping configured to:

- monitor validation loss;
- use patience `10`;
- restore the weights associated with the lowest validation loss.

Compare the reference and early-stopped runs using the number of completed epochs, best epoch, elapsed training time, learning curves, and validation metrics. Explain what early stopping changes and what it does not change. Discuss why monitoring validation loss can lead to a different stopping point than monitoring validation F1.

5. Compare activation functions

Train a `64, 64, 64` model with `tanh` hidden activations for 150 epochs without early stopping. Keep every other setting identical to the ReLU reference from Task 3.

Compare their learning curves, best epochs, and validation metrics. Explain why this is a controlled activation-function comparison and why the result does not imply that one activation is always superior.

6. Diagnose batch-size effects

Compare batch sizes `16`, `64`, and `256` using the unregularized `64, 64, 64` ReLU model, learning rate `0.001`, and 150 epochs without early stopping. Reuse the batch-size-64 reference from Task 3.

An optimizer update occurs once after each batch. Therefore, the number of optimizer updates per epoch is the number of batches, including a final incomplete batch: `ceil(number of training observations / batch size)`. Cumulative optimizer updates are this value multiplied by the number of completed epochs.

- Report the number of optimizer updates per epoch and the total number of updates in each run.
- Plot loss and F1 against both epoch and cumulative optimizer updates.
- Compare training time, convergence, curve variability, best epoch, and validation metrics.
- Explain why comparisons at the same epoch need not represent the same amount of optimization work.
- If a loss curve is highly variable, discuss whether batch size alone is a sufficient diagnosis. Identify at least two other plausible causes and propose one controlled follow-up experiment that could distinguish among them. You do not need to run the proposed follow-up.

Remember that an epoch-level loss is an aggregate across batches and can conceal some mini-batch variability. State clearly what quantity your plots display.

7. Compare regularization methods

Use the `64, 64, 64` ReLU architecture, batch size `64`, learning rate `0.001`, and 150 epochs without early stopping. Compare the unregularized reference from Task 3 with:

- L2 kernel regularization using `0.0001` on the kernel of each hidden `Dense` layer;
- L2 kernel regularization using `0.001` on the kernel of each hidden `Dense` layer;
- dropout with rate `0.25` after each hidden layer;
- dropout with rate `0.50` after each hidden layer.

L2 kernel regularization is not a separate layer. Apply the same L2 factor to the kernel of all three hidden `Dense` layers. Do not apply it to the biases or the output layer. For dropout, add three actual `Dropout` layers, one immediately after each hidden layer.

For every configuration:

- report the trainable parameter count, best epoch, elapsed training time, checkpoint training F1, and validation metrics;
- plot comparable training and validation loss and F1 curves;
- quantify the checkpoint training-minus-validation F1 gap and characterize the onset and severity of overfitting using the curves;
- explain whether the weaker and stronger setting of each method has too little, an appropriate, or too much effect;
- compare L2 regularization and dropout, including their effects on convergence and generalization.

Keras adds an L2 penalty to the loss optimized during training. Consequently, absolute loss values from an L2-regularized model and an unregularized model do not have exactly the same interpretation. Dropout is active during training and inactive during validation and prediction, which also affects how training and validation curves should be interpreted. Account for these facts in your discussion.

Combined L2 and dropout (optional, 2 bonus points)

Investigate whether combining L2 regularization and dropout provides complementary control or simply produces excessive regularization. Use the `64, 64, 64` ReLU architecture, batch size `64`, learning rate `0.001`, seed `4106`, 150 epochs, no early stopping, and the same checkpoint rule as the required experiments.

Train the four combinations in this `2 x 2` design:

	Dropout `0.25`	Dropout `0.50`
L2 `0.0001`	train	train
L2 `0.001`	train	train

Reuse the unregularized, L2-only, and dropout-only runs from Task 7. Report the same checkpoint and validation metrics for the four new models. Select the most informative combined configuration and plot it with the unregularized reference, the corresponding L2-only run, and the corresponding dropout-only run. Discuss whether the two methods appear complementary, redundant, or too strong when combined.

If you complete this experiment before accessing the test partition, the combined configurations are eligible for final selection in Task 8.

8. Select, refit, and test one final model

Create a compact table comparing all distinct configurations trained in Tasks 2 to 7 and, if completed, the optional combined configurations. Select exactly one configuration that you actually evaluated. Do not combine settings that were not tested together.

Use validation F1 at the lowest-validation-loss checkpoint as the primary selection criterion. Also consider precision, recall, ROC AUC, variability of the curves, parameter count, elapsed time, and the clarity of the diagnostic evidence. State the complete selected configuration and justify the choice.

The complete configuration includes the preprocessing, hidden layers and activation, L2 and dropout settings, optimizer and learning rate, batch size, output activation and loss, random seed, classification threshold, and locked checkpoint epoch.

Before examining the test partition:

1. record the selected run's checkpoint epoch;
2. instantiate and fit a fresh preprocessor on the complete development partition;
3. construct a fresh model with the selected architecture, activation, and regularization settings;
4. reset the neural-network seed to `4106`;
5. train on the complete development partition for the fixed number of epochs recorded in Step 1, without a validation partition or early stopping.

Evaluate this model once on the untouched common test partition. Report precision, recall, F1, ROC AUC, and a labelled confusion matrix. Compare these results with the validation estimate and with the saved test results for your final classical model from Assignment 2. Identify that classical model, but do not rerun or reselect it for this comparison. Discuss whether any difference is practically meaningful and identify at least one limitation of the comparison. The quality of your Assignment 2 model is not regraded here.

Do not revise the architecture, preprocessing, hyperparameters, regularization, threshold, or training duration after examining the test results.

9. Produce predictions for independent hidden missions

This task is worth 1 point for valid completion. Hidden-set performance is not used to determine the mark.

After completing Task 8, keep the selected configuration and locked checkpoint epoch unchanged. Then:

1. instantiate and fit a fresh preprocessor on all 6,371 labelled missions;
2. transform all labelled predictors with this preprocessor and prepare fresh, validated Keras arrays;
3. construct a fresh model with the selected configuration and reset the neural-network seed to `4106`;
4. train the model on all labelled missions for the locked number of epochs, without a validation partition or early stopping;
5. load the hidden file into a DataFrame named exactly `hidden_missions` and exclude `mission_id` from the model inputs;
6. transform the hidden predictors with the same final preprocessor and generate predicted probabilities and classes using threshold `0.5`;

7. create `asrc_missions_hidden_predictions.csv` with exactly these columns in this order:

```
mission_id,predicted_class,predicted_probability
```

8. save the probabilities without rounding, run the structural validator supplied in the template, and retain its output in the notebook.

The row order and `mission_id` values must match `hidden_missions`, `predicted_class` must contain only `0` or `1`, and `predicted_probability` must be complete, numeric, and between `0` and `1`. Each saved class must agree with its saved probability at threshold `0.5`. Submit the CSV with the notebook.

The supplied validator checks the file structure, mission IDs and row order, value ranges, and class-probability consistency. It does not use the hidden response and provides no information about predictive performance.

After the deadline, the teaching assistant will report hidden-set precision, recall, F1, ROC AUC, and a labelled confusion matrix. This feedback provides an independent check of generalization and connects the deployment workflow to Assignment 2. You receive the point for a documented final neural-network workflow and a structurally valid file, not for attaining a particular hidden-set score.

4. Documentation

Your Jupyter Notebook must include:

- your name, student number, and a descriptive report title;
- one clearly labelled section for each task;
- separate, logically sized cells for code and interpretation;
- a compact experiment table that makes the configurations and reused runs explicit;
- the output from validating `asrc_missions_hidden_predictions.csv`;
- enough explanation to make the experimental decisions and conclusions understandable;
- all execution results and graphics required for grading;
- a completed AI Use Declaration, whether or not you used AI assistance.

Evaluation (45 points)

- **Overall communication (3)**
- **Code quality and reproducibility (3)**
- **Neural-network study (37)**
  - Prepare and validate the neural-network inputs (3)
  - Connect logistic regression to a small neural network (4)
  - Study depth and capacity (6)
  - Evaluate early stopping (5)
  - Compare activation functions (2)
  - Diagnose batch-size effects (5)
  - Compare regularization methods (7)
  - Select, refit, and test one final model (4)
  - Produce predictions for independent hidden missions (1)
- **Resources, references, and AI declaration (2)**
- **Combined L2 and dropout (optional, up to 2 bonus points)**

Resources

The Keras documentation provides references for the `Sequential` model, `ModelCheckpoint`, `EarlyStopping`, layer weight regularizers, and `Dropout`. These references explain the APIs, but they do not replace your analysis of the assignment experiments.

Properly attribute any code or explanations drawn from external materials.

Do not list sources that you did not consult merely because they appear in this handout. If you did not use any external sources, write a brief statement such as "No external sources were used." You are not expected to add references merely to fill this section. The AI Use Declaration is still required in every submission.

Development strategy and further reading

Chollet and Karpathy provide practical discussions of the Overfit, Then Regularize strategy [1], [2]. Students who want to explore why highly overparameterized models can behave differently from the classical bias-variance account may also consult the optional articles on double descent [3], [4]. These articles provide broader theoretical context and are not required to complete the assignment.

[1] F. Chollet and M. Watson, *Deep Learning with Python*, 3rd ed. Shelter Island, NY, USA: Manning, 2026.

[2] A. Karpathy, "A recipe for training neural networks," Apr. 25, 2019. Accessed: Aug. 19, 2026. [Online]. Available: https://karpathy.github.io/2019/04/25/recipe/

[3] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling modern machine-learning practice and the classical bias-variance trade-off," *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, pp. 15849-15854, 2019, doi: 10.1073/pnas.1903070116.

[4] F. F. Yilmaz and R. Heckel, "Regularization-wise double descent: Why it occurs and how to eliminate it," in *2022 IEEE International Symposium on Information Theory (ISIT)*, Espoo, Finland, 2022, pp. 426-431, doi: 10.1109/ISIT50566.2022.9834569.

Reference style

Use **IEEE reference style**. Identify sources in the text using numbers in square brackets, such as `[1]` or `[2], [3]`, and list the references numerically in the order in which they are first cited. For online sources, include the page title, author or organization, access date, and URL. The IEEE Reference Guide contains additional examples.

Example in a written explanation:

> A staged development process separates deliberate overfitting from regularization [2].

When code is adapted from a source, place a citation in a nearby comment or Markdown cell:

```
# Adapted from the Keras documentation [5]
```

[5] Keras developers, "EarlyStopping," *Keras API documentation*. Accessed: Aug. 19, 2026. [Online]. Available: https://keras.io/api/callbacks/early_stopping/

Replace the example dates, titles, and URLs with information from the sources you actually used.

Use of artificial intelligence

AI assistance is permitted for this assignment, subject to the documentation requirements below. In contemporary software development, artificial intelligence is increasingly used by programmers to improve productivity, a trend that is likely to continue. Learning to use these tools critically and responsibly is therefore a valuable professional skill. Nevertheless, you must thoroughly understand the concepts and methods on which you are evaluated, particularly because AI tools may not be available during in-person assessments. You remain responsible for verifying, understanding, and being able to explain everything in your submission.

Permitted, properly declared AI use does not reduce your mark. Every submission must contain an **AI Use Declaration**, even if you did not use AI assistance.

1. **Did you use a generative AI tool while completing this assignment?** Answer Yes or No. This includes assistance with code, debugging, analysis, interpretation, writing, translation, or finding sources.

If you answer **No**, include the following confirmation and omit the remaining questions and the AI Interaction Transcript:

> I confirm that I did not use generative AI in preparing this submission.

If you answer **Yes**, answer the remaining questions:

2. **Which tools and models or versions did you use?**
3. **How did you use generative AI?** Select all that apply:
    - clarifying concepts or checking your understanding;
    - brainstorming or planning an approach;
    - writing or revising code;
    - debugging code or interpreting error messages;
    - interpreting data, results, tables, or plots;
    - writing or revising explanations;
    - translation;
    - finding references or documentation;
    - other uses, which you must specify.
4. **Briefly identify the sections or tasks for which AI assistance was used and describe how it influenced your work.**
5. **How did you verify the AI-generated suggestions, code, or explanations?**
6. Include the following confirmation:

    > I confirm that this declaration is complete, that the required transcript is included below, and that I understand and can explain everything in my submission.

If you used AI assistance, include the complete transcript of every assignment-related AI conversation. Most AI platforms retain conversation histories, so start a new conversation specifically for this assignment, use it consistently, and keep it focused on this work.

List each AI conversation or session in the numbered references. For example:

[6] OpenAI, “Conversation about diagnosing neural-network learning curves,” *ChatGPT*, model or version displayed by the interface, Nov. 10, 2026. Complete transcript included in the AI Interaction Transcript section.

Place the complete transcript in a separate **AI Interaction Transcript** section after the AI Use Declaration.

Questions

Post questions in the Assignment 3 discussion forum on Brightspace so that all students can benefit from the answers. If necessary, contact a teaching assistant directly.